NLP: Text Classification with SGD

Joanna Kołodziejczyk

December 2025

Context of the Assignment

This assignment is intended for an introductory NLP course. Its purpose is to show how a short text can be transformed into a numerical feature vector and then classified by a neural model trained with stochastic gradient descent (SGD). The task concerns binary text classification, for example sentiment classification, where:

$$y = 1 \quad \text{means a positive text,} \qquad y = 0 \quad \text{means a negative text.}$$

The model output is interpreted as:

$$\hat{y} = P(y = 1 \mid d),$$

where d denotes the input document.

For simplicity, the calculations are performed for one training example only. In a real NLP experiment, the model would be trained on many documents, with a separate validation or test set.

1 SGD for a Single-Neuron Text Classifier – Two Learning Steps

Consider a single-neuron classifier, equivalent to logistic regression for binary text classification:

$$\hat{y} = \sigma(z), \quad z = w_1 x_1 + w_2 x_2 + b.$$

The sigmoid activation function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Text Representation

Let the training document be:

$$d = \text{“excellent excellent but slightly boring”}.$$

After lowercasing, tokenization, and removing punctuation, represent the document using a small bag-of-words vocabulary:

$$V = \{\text{excellent}, \text{boring}\}.$$

Use two features:

$$x_1 = \text{count(excellent in } d), \qquad x_2 = \text{count(boring in } d).$$

For the document above:

$$x_1 = 2, \quad x_2 = 1, \quad y = 1.$$

Thus, the text is represented by the vector:

$$\mathbf{x} = (2, 1).$$

Task: Explain briefly why a text must be converted into a numerical vector before it can be used by a neural classifier. Then plot the point $(x_1, x_2) = (2, 1)$ in the two-dimensional feature space. The horizontal axis should correspond to the count of the word `excellent`, and the vertical axis should correspond to the count of the word `boring`.

Initial Weights

At the beginning of training, the model parameters are initialized as follows:

$$w_1 = 0.5, \quad w_2 = -0.5, \quad w_0 = b = 0.$$

In this simplified text-classification example, w_1 measures the influence of the word `excellent` on the positive class, while w_2 measures the influence of the word `boring` on the positive class.

Task: Draw the decision boundary represented by the initial weights:

$$w_1 x_1 + w_2 x_2 + b = 0.$$

Interpret the signs of w_1 and w_2 in terms of sentiment classification.

Loss Function

Use the binary cross-entropy loss:

$$L(y, \hat{y}) = -\left(y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})\right).$$

It measures the discrepancy between:

- the true class label $y \in \{0, 1\}$,
- the predicted probability $\hat{y} \in (0, 1)$ generated by the classifier.

The loss function strongly penalizes low values of $\hat{y}$ when $y = 1$ and high values of $\hat{y}$ when $y = 0$. Therefore, it is appropriate for probabilistic binary text classification.

Special Cases

For $y = 1$:

$$L = -\ln(\hat{y}).$$

For $y = 0$:

$$L = -\ln(1 - \hat{y}).$$

Illustrative Numerical Example

If $y = 1$ and $\hat{y} = 0.38$, then:

$$L = -\ln(0.38) \approx 0.97.$$

This means that the classifier assigns too low a probability to the correct positive class. The loss is relatively large, so the update should change the weights in a direction that increases $\hat{y}$ for similar positive texts.

For comparison, if $\hat{y} = 0.95$ and $y = 1$, then:

$$L = -\ln(0.95) \approx 0.05,$$

which indicates a very good prediction and only a small correction of the weights.

Learning Algorithm Parameter

Table 1: Learning algorithm parameter

Method	Parameter
SGD	$\eta = 0.1$

Instructions

1. **Forward propagation.** Calculate:
$$z = w_1 x_1 + w_2 x_2 + b$$
for the text vector $\mathbf{x} = (2, 1)$, and then calculate:
$$\hat{y} = \sigma(z).$$
Interpret $\hat{y}$ as the predicted probability that the text is positive.

2. **Loss value.** Calculate the value of the loss function $L(y, \hat{y})$.

3. **Output error and gradients.** Define the error at the classifier output:
$$\delta_t = \frac{\partial L}{\partial z} = \hat{y} - y.$$
The gradients with respect to the parameters are:
$$g_{w_1,t} = \frac{\partial L}{\partial w_1}, \qquad g_{w_2,t} = \frac{\partial L}{\partial w_2}, \qquad g_{b,t} = \frac{\partial L}{\partial b}.$$
Determine them using:
$$g_{w_1,t} = \delta_t x_1 = (\hat{y} - y)x_1,$$
$$g_{w_2,t} = \delta_t x_2 = (\hat{y} - y)x_2,$$
$$g_{b,t} = \delta_t = \hat{y} - y.$$
Explain which feature, `excellent` or `boring`, causes the larger absolute gradient and why.

4. **Parameter update using SGD.** Update the parameters using:
$$w_{\text{SGD}}^{(t)} = w^{(t-1)} - \eta g_t.$$
Calculate separately:
$$w_1^{(t)} = w_1^{(t-1)} - \eta g_{w_1,t},$$
$$w_2^{(t)} = w_2^{(t-1)} - \eta g_{w_2,t},$$
$$b^{(t)} = b^{(t-1)} - \eta g_{b,t}.$$
Interpret how the update changes the importance of the two words.

Final Results to Report After the First Learning Step

Report the results after the first SGD step. Draw the new decision boundary resulting from the corrected weight values and explain whether the classifier becomes more confident for the training text.

Parameter	SGD
$w_1^{(1)}$	
$w_2^{(1)}$	
$b^{(1)}$	

Second SGD Iteration ($t = 2$)

Repeat the same procedure using the parameters obtained after the first SGD step.

1. **Forward propagation using the parameters obtained after the first SGD step:**

$$z_{\text{SGD}}^{(1)}, \quad \hat{y}_{\text{SGD}}^{(1)}.$$

2. **Calculation of the loss function:**

$$L_{\text{SGD}}^{(1)}.$$

3. **Determination of the output error:**

$$\delta_{2,\text{SGD}} = \hat{y}_{\text{SGD}}^{(1)} - y.$$

4. **Determination of the gradients:**

$$g_{w_1,2}^{\text{SGD}} = \delta_{2,\text{SGD}} x_1, \quad g_{w_2,2}^{\text{SGD}} = \delta_{2,\text{SGD}} x_2, \quad g_{b,2}^{\text{SGD}} = \delta_{2,\text{SGD}}.$$

5. **Parameter update using SGD, step $t = 2$:**

$$\theta_{\text{SGD}}^{(2)} = \theta_{\text{SGD}}^{(1)} - \eta g_{\theta,2}^{\text{SGD}}, \quad \theta \in \{w_1, w_2, b\}.$$

Final Results to Report After the Second Learning Step

Report the results after the second SGD step: Draw the new decision boundary and discuss the change in

Parameter	SGD
$w_1^{(2)}$	
$w_2^{(2)}$	
$b^{(2)}$	

the predicted probability of the positive class.

2 SGD for a Neural Text Classifier with Architecture 2–2–1

Now consider a small neural network for the same binary text-classification task. The network has the structure:

$$2 \ \rightarrow \ 2 \ \rightarrow \ 1,$$

with a sigmoid activation function in both the hidden layer and the output layer.

The input layer consists of the two bag-of-words features:

$$x_1 = \text{count}(\texttt{excellent}), \qquad x_2 = \text{count}(\texttt{boring}).$$

The two hidden neurons may be interpreted as intermediate detectors that combine lexical evidence before the final sentiment decision is made.

Initial Weights

Hidden layer with two neurons:

Neuron 1:

$$w_{11}^{(0)} = 0.2, \quad w_{12}^{(0)} = -0.1, \quad b_1^{(0)} = 0.00.$$

Neuron 2:

$$w_{21}^{(0)} = 0.4, \quad w_{22}^{(0)} = 0.3, \quad b_2^{(0)} = 0.00.$$

Output layer with one neuron:

$$v_1^{(0)} = 0.5, \quad v_2^{(0)} = -0.4, \quad b_3^{(0)} = 0.00.$$

Use the same training document vector:

$$\mathbf{x} = (2, 1), \qquad y = 1.$$

Perform the following steps:

1. **Forward propagation.**

   Hidden layer:

   $$z_1^{(t)} = w_{11}^{(t)} x_1 + w_{12}^{(t)} x_2 + b_1^{(t)}, \quad h_1^{(t)} = \sigma(z_1^{(t)}).$$
   $$z_2^{(t)} = w_{21}^{(t)} x_1 + w_{22}^{(t)} x_2 + b_2^{(t)}, \quad h_2^{(t)} = \sigma(z_2^{(t)}).$$

   Output layer:

   $$z_3^{(t)} = v_1^{(t)} h_1^{(t)} + v_2^{(t)} h_2^{(t)} + b_3^{(t)}, \quad \hat{y}^{(t)} = \sigma(z_3^{(t)}).$$

   Interpret $\hat{y}^{(t)}$ as the probability that the document is classified as positive.

2. **Calculation of the loss function.**

   $$L^{(t)} = -\left(y \ln \hat{y}^{(t)} + (1 - y) \ln\bigl(1 - \hat{y}^{(t)}\bigr) \right).$$

3. **Determination of the error at the network output.**

   $$\delta_{3,t} = \frac{\partial L}{\partial z_3^{(t)}} = \hat{y}^{(t)} - y.$$

4. **Gradients of the output layer.**

   $$g_{v_1,t} = \frac{\partial L}{\partial v_1} = \delta_{3,t} h_1^{(t)}, \quad g_{v_2,t} = \frac{\partial L}{\partial v_2} = \delta_{3,t} h_2^{(t)}.$$

   $$g_{b_3,t} = \frac{\partial L}{\partial b_3} = \delta_{3,t}.$$

5. **Errors of the hidden-layer neurons.**

   $$\delta_{1,t} = \delta_{3,t} v_1^{(t)} h_1^{(t)} \bigl(1 - h_1^{(t)}\bigr).$$

   $$\delta_{2,t} = \delta_{3,t} v_2^{(t)} h_2^{(t)} \bigl(1 - h_2^{(t)}\bigr).$$

6. **Gradients of the hidden layer.**

   $$g_{w_{11},t} = \delta_{1,t} x_1, \quad g_{w_{12},t} = \delta_{1,t} x_2.$$
   $$g_{w_{21},t} = \delta_{2,t} x_1, \quad g_{w_{22},t} = \delta_{2,t} x_2.$$
   $$g_{b_1,t} = \delta_{1,t}, \quad g_{b_2,t} = \delta_{2,t}.$$

   Explain how the text features influence the gradients in the hidden layer.

7. **Parameter update using SGD.**

   $$\theta_{\mathrm{SGD}}^{(t+1)} = \theta_{\mathrm{SGD}}^{(t)} - \eta g_{\theta,t}, \quad \theta \in \{w_{11}, w_{12}, w_{21}, w_{22}, v_1, v_2, b_1, b_2, b_3\}.$$

8. **Report the updated parameter values obtained using SGD.**

   $$\{w_{ij}^{(t+1)}, v_k^{(t+1)}, b_\ell^{(t+1)}\}_{\mathrm{SGD}}.$$

   State whether the update increases or decreases the predicted probability of the positive class for the training text.

Scoring and Submission Format

Points are awarded for the correct completion of the individual stages of the assignment as follows:

Assignment component	**Points**
Text preprocessing, vocabulary construction, and bag-of-words representation	10
Single-neuron classifier: forward propagation and loss function	10
Single-neuron classifier: output error and gradients	10
Single-neuron classifier: first update step using SGD	10
Single-neuron classifier: second learning step using SGD	10
Feature-space decision boundary and interpretation of word weights	10
2–2–1 neural text classifier: forward propagation and loss function	15
2–2–1 neural text classifier: backpropagation errors and gradients	10
2–2–1 neural text classifier: SGD update	10
NLP interpretation and conclusions	5
Total	**100**

The calculations may be performed:

- by hand on paper, submitted as a scan or photograph in PDF format, or
- using a computational script. The results should be saved as `report.pdf`, containing all calculations presented step by step, with the stages clearly separated.

The report should clearly show:

- the preprocessing steps: lowercasing, tokenization, punctuation removal, and vocabulary selection,
- the bag-of-words vector for the given text,
- forward propagation,
- calculation of the error and gradients,
- SGD updates,
- the feature-space graph for the single-neuron classifier,
- each calculation step written separately,
- final conclusions.

The following points should be briefly discussed:

- why text must be represented numerically before classification,
- how the weights associated with `excellent` and `boring` change after SGD updates,
- how the loss function changes after each SGD step,
- how the decision boundary shifts in the two-dimensional feature space,
- why this example is pedagogical and how it would be extended to a realistic NLP dataset with a larger vocabulary and a train/test split.